MINI PROJECT REPORT

MINIMUM NUMBER OF EV CHARGING STATIONS FOR A COVERAGE TARGET USING BINARY SEARCH ON THE ANSWER

Submitted by:

Easala Jagadeesh

2411021061183

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

Dr. Shelly Shiju George

Associate Professor, ASAC

Alliance School of Advanced Computing

Alliance University, Bengaluru

October-2026



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on **Design and Analysis of Algorithms** is submitted by **Easala Jagadeesh** [Uni. Reg. No. 2411021061183] in partial fulfilment for the award of the degree of Bachelor of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026-27. This report embodies the results of original work and studies conducted by the student and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

- - - - - - - - - - - - - - - - - - - -

Dr. Shelly Shiju George

Associate Professor



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled **MINIMUM NUMBER OF EV CHARGING STATIONS FOR A COVERAGE TARGET USING BINARY SEARCH ON THE ANSWER**, submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (CSE) at Alliance University, is a record of my original work carried out under the supervision of Dr. Shelly Shiju George, Associate Professor.

I further confirm that all views, findings, and conclusions expressed herein are the result of sincere effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

Easala Jagadeesh	**2411021061183**	
Name of the Student	University Registration Number	Signature

Acknowledgement

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this project titled **MINIMUM NUMBER OF EV CHARGING STATIONS FOR A COVERAGE TARGET USING BINARY SEARCH ON THE ANSWER**.

First and foremost, I extend my sincere thanks to my mentor and guide, Dr. Shelly Shiju George, for the unwavering support, insightful feedback, and encouragement throughout the development of this system. The guidance received in the subject Design and Analysis of Algorithms provided the perfect foundation for this work.

I am deeply grateful to my faculty members and peers whose guidance and technical inputs helped refine the project. Their collaborative spirit and constructive suggestions were instrumental in shaping the final outcome.

This project allowed me to apply the binary search technique from the course to a real planning question, to analyse its cost with recurrences and asymptotic notation, and to confirm the analysis with measurements. It strengthened my understanding of how the right algorithm design saves real time.

Lastly, I would like to thank everyone who supported me directly or indirectly during this journey. Your encouragement and belief in my work made this accomplishment possible.

Thank you.

Easala Jagadeesh

TABLE OF CONTENTS

Acknowledgement **4**

Abstract **6**

Chapter I- Introduction **7**

Problem Statement 7

Theory 7

Chapter II - System Configuration **9**

Algorithm 9

Source Code 9

Chapter III - Results **11**

Finding the minimum for a 50% target 11

The question is monotone 12

Binary search compared with linear search 13

Chapter IV - Conclusion **15**

Abstract

A city planning electric-vehicle charging stations must answer a budget question before it can place any station: what is the smallest number of stations that serves a target share of the city's charging demand, for example 50%? Checking every possible number of stations one by one is slow, because every check requires solving a hard placement problem.

This report applies binary search on the answer to this question. Because adding a station can never reduce the demand covered, the answer to 'are k stations enough?' is false for small k and true from some point on. Binary search finds that point by halving the range at every step, using $\Theta(\log n)$ checks instead of the $\Theta(n)$ checks of a linear search. Each check runs a simulated-annealing optimizer, since an exact search over, for example, the 16,007,560,800 ways of placing seven stations is infeasible.

For a 50% target the search found the answer, 7 stations, in 5 checks; for the range 1 to 20 it never needs more than 5, while a linear search needed up to 20 checks. Measured running times show a trade-off: binary search always uses at most 5 checks, but because a check with many stations costs more than one with few, a linear search can still be quicker when the answer is small.

Chapter I- Introduction

Design and Analysis of Algorithms is concerned with two questions: how to design a correct procedure for a problem, and how its running time grows with the size of the input. Divide and conquer is one of the main design paradigms, and binary search is its simplest example: it solves a problem of size n by solving one problem of half the size.

Binary search is usually taught on a sorted array, but the same idea works on any yes/no question whose answer changes only once over a range of values. This is called binary search on the answer. This report applies it to a planning question from the EV Charge Planner project.

Problem Statement

The city is a 10 × 10 grid of 100 zones whose daily charging demand (total 1338.4 sessions per day) has been predicted from survey data. A station serves its own zone and the four zones next to it. The problems addressed are:

- to find the smallest number of stations k that can cover a given share of the city's demand;
- to show that the question 'are k stations enough?' is monotone, so that binary search applies;
- to analyse the cost of the search with a recurrence and compare it with a linear search; and
- to measure the number of checks and the running time of both searches.

Theory

Binary search and its correctness

Binary search keeps an interval [low, high) that is guaranteed to contain the answer. At each step it tests the middle value mid: if mid is enough, the answer is mid or smaller, so high = mid; otherwise the answer is larger, so low = mid + 1. The *loop invariant* 'the smallest k that is enough lies in [low, high]' is true at the start and is kept by both updates, and the interval shrinks every step, so the loop ends with low equal to the answer.

Recurrence and running time

Each step does a constant amount of work besides the check and halves the range, so the number of steps satisfies

$$\mathbf{T(n) = T(n / 2) + O(1)}$$

By the Master Theorem ($a = 1$, $b = 2$, $f(n) = O(1) = \Theta(n^{\log_2 1})$, case 2) this gives $T(n) = \Theta(\log n)$. For the range 1 to 20 (21 possible outcomes including 'not possible') binary search needs at most $\lceil \log_2 21 \rceil = 5$ checks, while a linear search that tries k = 1, 2, 3, ... needs up to 20.

Binary search on the answer

Let enough(k) be true when k stations can cover the target. Adding a station can never reduce the covered demand, because the extra station can always be placed where it adds nothing new. Therefore enough(k) is *monotone*: false, false, ..., false, true, true, ... This is exactly the property binary search needs, the same as in the 'Koko eating bananas' problem, where a faster eating speed is always enough if a slower one is.

The cost of one check

Number of stations k	Possible placements C(100, k)
3	161,700
5	75,287,520
7	16,007,560,800
10	17,310,309,456,440

Table 1: Why a check cannot be done by brute force

Deciding enough(k) exactly would require trying every placement of k stations, which is impossible for k above 3 or 4 (Table 1). Each check therefore runs the project's simulated-annealing optimizer twice (4,000 steps each) and keeps the better result. A check costs C operations, so the whole search costs Θ(C log n) instead of Θ(C n): binary search saves expensive checks, which is where the time goes.

Chapter II - System Configuration

The binary search is part of the EV Charge Planner project, written from scratch in Python 3 without external libraries. It can be run from the terminal or from the project's web interface, where a slider selects the coverage target.

Component	Specification
Laptop	Dell Inspiron 15 3525
Processor	AMD Ryzen 5 5625U with Radeon Graphics, 6 cores / 12 threads
Memory	8 GB RAM
Operating system	Windows 11 Home Single Language
Language	Python 3.12 (standard library only, no external libraries)
Web interface	HTML5, CSS3, JavaScript (ES6), served by Python's built-in http.server
Browser	Microsoft Edge (Chromium)
Version control	Git and GitHub

Table 2: System configuration

Algorithm

Binary search for the minimum number of stations

1. Set goal = target share × total demand, low = 1 and high = 20 + 1 (high = 21 stands for 'not possible').
2. While low < high: set mid = (low + high) / 2, rounded down.
3. Run the check: the best coverage found for mid stations (two simulated-annealing runs).
4. If the coverage reaches the goal, set high = mid; otherwise set low = mid + 1. Record the step.
5. When the loop ends, low is the answer, or 'not possible' if low = 21.

Linear search (for comparison)

1. For k = 1, 2, 3, ... run the same check.
2. Stop at the first k whose coverage reaches the goal.

Source Code

The listing below is taken directly from the project file main.py.

Binary search on the answer

```
def best_coverage(demand, k):
    """Returns coverage achieved with k stations using simulated annealing."""
    return max(score(simulated_annealing(demand, steps=4000, k=k), demand) for _ in range(2))

def min_stations_needed(demand, target_share):
    """
    Binary searches the range [1, MAX_STATIONS] for the minimum station count.
```

```
Returns (answer, step_history).
"""
goal = target_share * sum(demand.values())
low, high = 1, MAX_STATIONS + 1
steps = []
while low < high:
    mid = (low + high) // 2
    covered = best_coverage(demand, mid)
    enough = covered >= goal
    steps.append((mid, covered, enough))
    if enough:
        high = mid
    else:
        low = mid + 1
return (low if low <= MAX_STATIONS else None), steps
```

Chapter III - Results

Finding the minimum for a 50% target

Table 3 shows the steps of the binary search for a target of 50% of the city's demand (669.2 of 1338.4 sessions per day).

Check	Stations tried (mid)	Coverage	Decision
1	11	68.7%	enough → try fewer
2	6	46.9%	not enough → try more
3	9	60.7%	enough → try fewer
4	8	56.7%	enough → try fewer
5	7	51.6%	enough → try fewer

Table 3: Steps of the binary search (target 50%)

The answer is **7 stations**, found in 5 checks. A linear search would have needed 7 checks to reach the same answer, and up to 20 for a high target.

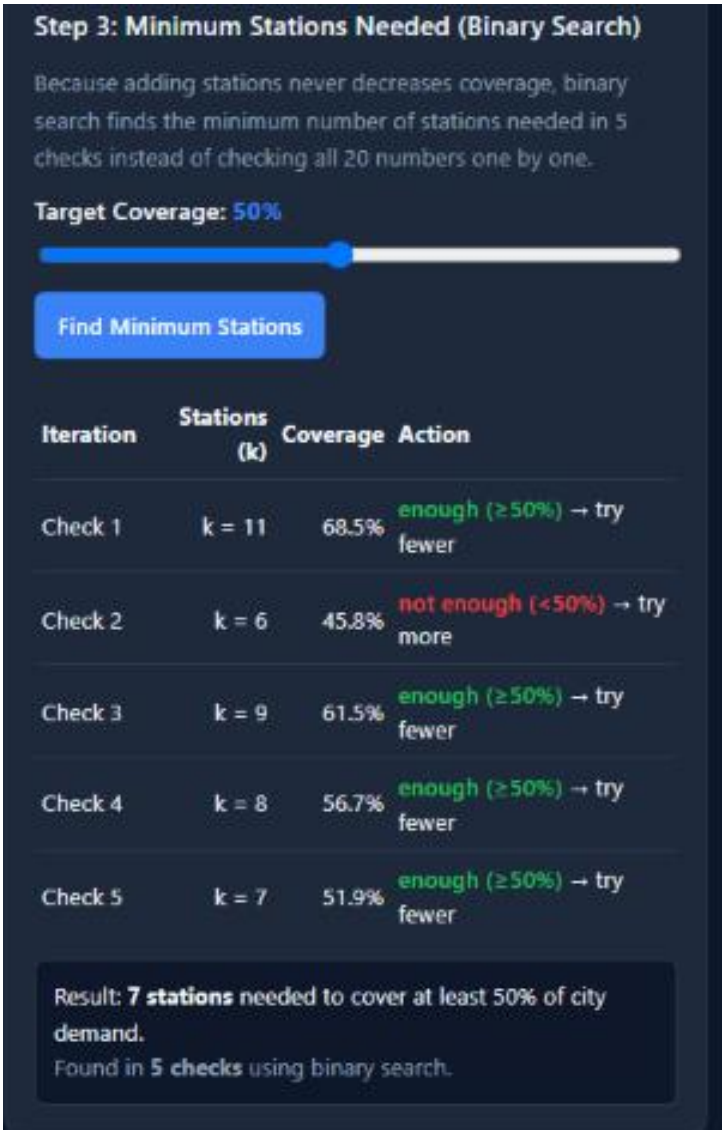


Figure 1: The binary search in the web interface. Coverage percentages differ slightly between runs because each check uses randomised annealing; the answer is the same

The question is monotone

To confirm that binary search is valid, the best coverage was computed for every k from 1 to 20 (Figure 2), from 12.0% with one station to 89.9% with 20. Coverage rises steadily with the number of stations; the only exception is a small dip of 0.0 percentage points between 19 and 20 stations. The true best coverage can never fall when a station is added, so this dip comes from the randomised optimizer finding a slightly worse plan for the larger k, not from the problem itself.

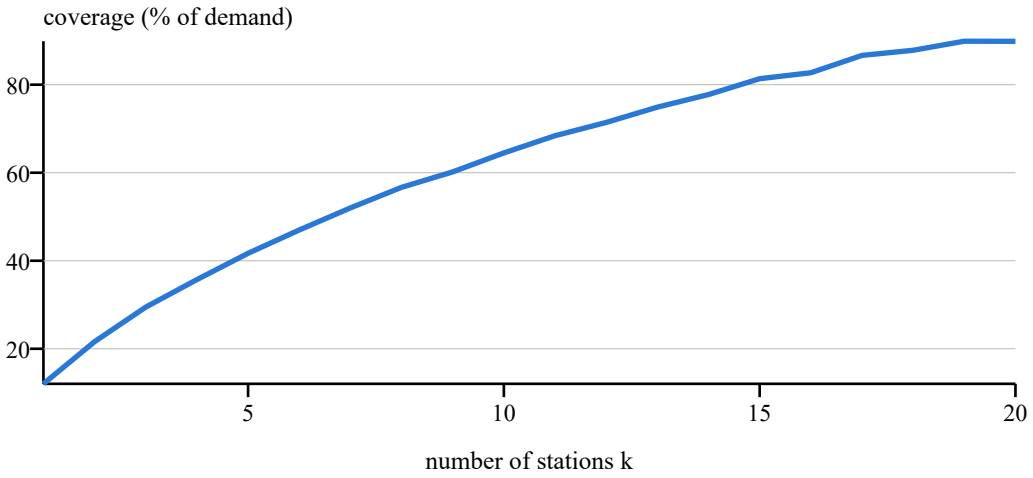


Figure 2: Best coverage found for each number of stations

Binary search compared with linear search

Both searches were run for three targets with the same random seed, counting the checks and measuring the total time (Table 4).

Target	Answer	Binary search checks	Binary time (s)	Linear search checks	Linear time (s)
30%	4	5	1.8	4	0.6
50%	7	5	2.5	7	1.8
70%	12	5	4.3	12	4.9
85%	18	5	5.0	18	11.9
95%	not possible	4	5.3	20	16.8

Table 4: Checks and running time of binary and linear search

Binary search used at most 5 checks for every target, as the analysis predicts, while the number of checks of the linear search grew with the answer, up to 20 for the 95% target. In running time the picture is mixed. Linear search was faster for the 30%, 50% target(s): it only tries small numbers of stations, whose checks are cheap, while binary search starts in the middle of the range with expensive checks. Binary search was faster for the 70%, 85%, 95% target(s), where the linear search has to try many large numbers of stations.

The reason is that the checks do not all cost the same: a check with k stations runs the optimizer on k stations, so its cost grows with k. The $\Theta(\log n)$ analysis counts checks, and it is the right measure when checks cost about the same or when the answer may lie anywhere in the range. When the answer is usually small and checks get more expensive as k grows, a search that starts from small k (for example exponential search: 1, 2, 4, 8, ... followed by binary search) combines the advantages of both. Note also that each check uses a randomised

optimizer, which finds the best plan almost always but not with a mathematical guarantee, so the answer is a very good estimate rather than a proof.

Chapter IV - Conclusion

This report applied binary search on the answer to a real planning question: the smallest number of charging stations that covers a target share of a city's demand. The key observation is that the question is monotone, which was confirmed by computing the coverage for every number of stations. With that property, binary search found the answer for a 50% target, 7 stations, in 5 checks.

The recurrence $T(n) = T(n/2) + O(1)$ gives $\Theta(\log n)$ checks, against $\Theta(n)$ for a linear search, and the measurements confirmed that binary search never needed more than 5 checks while a linear search needed up to 20. The running times also showed that counting checks is not the whole story: when a check becomes more expensive as k grows, a linear search from small k can be quicker for small answers. Choosing an algorithm therefore needs both the asymptotic analysis and an understanding of the real cost of each step.

Future work includes exponential search, which starts with small k and then switches to binary search, an exact check for small instances using dynamic programming or integer programming, and reusing the placement from one check as the starting point of the next to make each check faster.